

PORTADA

Calidad de Software

Semana 2 — Verificación, validación y el proceso de aseguramiento

Universidad de Antioquía · Ingeniería de Sistemas



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 1

- **Error, defecto y falla** son tres cosas distintas: la persona, el artefacto y el comportamiento.
- **Crosby**: calidad = conformidad con los requisitos. **Juran**: calidad = adecuación al uso.
- La calidad vive en tres lugares: **producto** (ISO/IEC 25010), **proceso** (ISO/IEC 12207, CMMI) y **uso**.
- Premisa de cierre: **la calidad del proceso condiciona la del producto**.



TRANSICIÓN

Hoy: el proceso

Si la calidad del proceso condiciona la del producto, hace falta un nombre para "cuidar el proceso": eso es el **aseguramiento de la calidad**.

Dentro de ese aseguramiento hay dos preguntas que se confunden todo el tiempo — y un modelo clásico que las organiza — y un debate sobre quién debe hacerlas.



CONCEPTO

Qué es el aseguramiento de la calidad (QA)

QA — *Quality Assurance* — es, según el estándar IEEE, un **conjunto planeado y sistemático de actividades** que da confianza en que el proceso y sus productos cumplen los requisitos técnicos y de gestión establecidos.

No es "probar el software al final": es un conjunto de actividades distribuidas a lo largo de todo el proceso — estándares, revisiones, auditorías, capacitación — que buscan **evitar** que el defecto llegue a existir, no solo encontrarlo.



CONCEPTO

QA no es QC

Son dos siglas que se usan como sinónimos y no lo son. El curso distingue el **aseguramiento** (QA) del **control** (QC) de calidad:

	QA — Aseguramiento	QC — Control
Enfoque	El proceso	El producto
Naturaleza	Preventiva	Detectiva
Pregunta	¿Estamos siguiendo un buen proceso?	¿Este artefacto tiene defectos?
Ejemplos	Estándares, revisiones de diseño, capacitación	Pruebas, inspecciones de código, auditorías de producto

El testing es una actividad de QC que ocurre dentro de un programa de QA — no es lo mismo que todo el programa.

CONCEPTO

El ciclo de aseguramiento: Planificar–Hacer–Verificar–Actuar

Walter Shewhart lo propuso y Deming lo popularizó como **PDCA** (*Plan–Do–Check–Act*). Es el mismo espíritu que la trilogía de Juran de la semana pasada, aplicado como ciclo que se repite:

- **Planificar:** definir el estándar o proceso a seguir y qué se espera lograr.
- **Hacer:** ejecutar el proceso a pequeña escala o en el trabajo real.
- **Verificar:** comparar el resultado obtenido contra lo planeado.
- **Actuar:** ajustar el proceso con lo aprendido, y volver a planificar.



TRANSICIÓN

Dos preguntas distintas dentro del aseguramiento

El ciclo PDCA tiene un paso que dice "Verificar" — pero en aseguramiento de calidad de software esa palabra se separa en dos, porque responden preguntas distintas y se confunden constantemente en la práctica.



CONCEPTO

Verificación: ¿lo estamos construyendo correctamente?

Evalúa si los **artefactos de una fase** —un diseño, un módulo de código, un documento— cumplen las condiciones que se impusieron al comenzar esa fase. Es una comprobación de **conformidad interna**.

- Técnicas típicas: revisiones de código, inspecciones, walkthroughs, análisis estático.
- Ocurre **durante** el desarrollo, fase por fase — no hace falta que el sistema completo esté ejecutándose.
- Responde: ¿esto respeta la especificación que se le dio?



CONCEPTO

Validación: ¿estamos construyendo lo correcto?

Evalúa si el **producto** —normalmente ya en ejecución, completo o parcial— satisface la **necesidad real** de quien lo va a usar. Es una comprobación contra el mundo, no contra un documento.

- Técnicas típicas: pruebas funcionales, pruebas de aceptación, demostraciones con usuarios reales.
- Ocurre normalmente **al final** de una fase o del proyecto, cuando hay algo ejecutable que mostrar.
- Responde: ¿esto resuelve el problema que el usuario tiene?



ORIGEN

Barry Boehm, 1979

Boehm es ingeniero de software, entonces en TRW Defense Systems, y más tarde una de las figuras centrales en la creación del modelo espiral de desarrollo. En 1979 publicó un artículo sobre verificación y validación de software que dejó la formulación que hasta hoy se enseña en toda introducción al tema.

No inventó los conceptos —ya se usaban en ingeniería de sistemas antes que en software— pero les dio la frase que los volvió memorables y que separa, de una vez, cualquier confusión entre ambos:

"Verification: are we building the product right? Validation: are we building the right product?"

CONCEPTO

Verificación y validación, lado a lado

	Verificación	Validación
Pregunta	¿Lo construimos correctamente?	¿Construimos lo correcto?
Se mide contra	La especificación	La necesidad real del usuario
Cuándo	Durante cada fase	Con el producto ejecutándose
Técnicas	Revisiones, inspecciones, análisis estático	Pruebas funcionales, aceptación, demos
Puede fallar aunque...	...el usuario esté satisfecho	...la especificación se haya cumplido al pie de la letra

APLICADO

El sistema de matrícula, otra vez

La semana pasada usamos este ejemplo para hablar de Crosby y Juran. Con el vocabulario de hoy, se dice más preciso:

- **Verificación: aprobada.** El sistema conforma con el 100 % de lo especificado.
- **Validación: fallida.** Ningún estudiante logra matricularse sin ayuda.

Se puede verificar con éxito y validar con fracaso al mismo tiempo — son ejes independientes, no dos nombres de lo mismo.



INTERACCIÓN

Ejercicio rápido: ¿verificación o validación?

Clasifica cada actividad — 5 minutos, en el chat o en voz alta:

1. Un compañero revisa tu código antes de fusionarlo a la rama principal.
2. Un grupo de usuarios reales prueba la app y da su opinión sobre si les sirve.
3. Se compara el diseño de la base de datos contra el documento de requisitos.
4. El cliente aprueba la funcionalidad entregada al final del sprint.

Esta dinámica es un calentamiento para la matriz del taller de hoy.

CONCEPTO

El modelo en V

Es un modelo de ciclo de vida que **empareja** cada fase de desarrollo con su fase de prueba correspondiente, definida desde el principio — no al final, como una ocurrencia tardía.

Fase de desarrollo	Fase de prueba correspondiente
Requisitos de usuario	Pruebas de aceptación
Especificación funcional / diseño arquitectónico	Pruebas de sistema
Diseño detallado	Pruebas de integración
Codificación	Pruebas unitarias

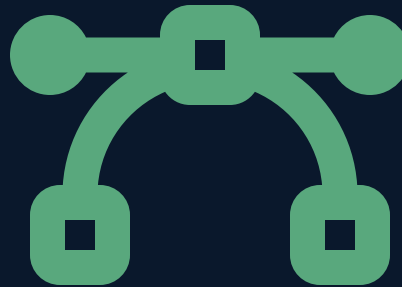
La codificación está en la punta de la "V", uniendo la rama de desarrollo (baja) con la de pruebas (sube).

CONCEPTO

Qué resuelve el modelo en V

Cada prueba se **diseña en la misma fase que su contraparte de desarrollo**, no después de escribir el código:

- Los criterios de aceptación se escriben junto con los requisitos, no al entregar.
- Si un requisito no tiene un criterio de aceptación claro, **la prueba correspondiente no se puede diseñar** — el hueco queda visible temprano.
- Fuerza a pensar en cómo se va a probar algo antes de construirlo.



CONCEPTO EN DISPUTA

Una crítica común al modelo en V

El modelo en V asume que los requisitos quedan fijos desde el principio y que las fases ocurren una sola vez, en secuencia. En proyectos ágiles, donde los requisitos cambian sprint a sprint, esa suposición no se sostiene.

La alternativa habitual no es abandonar la idea de emparejar desarrollo con pruebas, sino **repetir ese emparejamiento en cada iteración** en vez de una sola vez al inicio del proyecto.

El modelo en V no está "mal": describe bien un proyecto con requisitos estables. El problema aparece cuando se aplica sin cuestionar a un contexto que no lo es.



TRANSICIÓN

Ahora: quién hace esto

Ya sabemos qué preguntas responder y en qué momento del ciclo. Falta la última pieza: quién es responsable de responderlas dentro de un equipo real.



CONCEPTO

Roles típicos en un equipo de QA

- **QA Lead / Manager:** define la estrategia de calidad del proyecto, los estándares a seguir y coordina al resto del equipo de QA.
- **Analista / diseñador de pruebas:** convierte requisitos en casos de prueba concretos — el rol más cercano a lo que hará el Trabajo 1 de este curso.
- **Tester manual:** ejecuta pruebas exploratorias y casos que aún no vale la pena automatizar.
- **Ingeniero de automatización:** a veces llamado **SDET** (*Software Development Engineer in Test*) — escribe código para automatizar pruebas repetitivas.

En equipos pequeños, una sola persona cubre varios de estos roles a la vez.

CONCEPTO

QA en equipos ágiles: responsabilidad compartida

En metodologías ágiles, el tester no es una **puerta al final** del proceso: se integra al equipo desde el primer día del sprint, participa en la planeación y ayuda a escribir los criterios de aceptación.

La calidad deja de ser trabajo exclusivo de "el área de QA" y pasa a ser **responsabilidad de todo el equipo** — desarrolladores incluidos.

Conecta con Knight Capital: faltó justo eso — que una segunda persona revisara el despliegue antes de que saliera a producción.



CONCEPTO EN DISPUTA

¿Hace falta un rol de QA dedicado?

Si la calidad es "responsabilidad de todo el equipo", ¿para qué contratar testers dedicados? Un argumento fuerte en contra de eliminarlos: quien escribe el código tiende a probar lo que **cre**e que construyó, no lo que realmente construyó — sus propios supuestos son invisibles para sí mismo.

El contraargumento: un rol separado puede volverse, otra vez, una puerta al final que retrasa la entrega y le quita presión al desarrollador para escribir código correcto desde el principio.



CONCEPTO

Niveles de independencia de las pruebas

1. **Sin independencia:** la misma persona que escribió el código prueba su propio trabajo.
2. **Independencia dentro del equipo:** otro desarrollador del mismo equipo prueba el código.
3. **Equipo de pruebas independiente:** un equipo separado dentro de la misma organización.
4. **Independencia externa:** una organización distinta certifica el producto.

A mayor independencia, menos sesgo pero menos contexto y más fricción de coordinación. No hay un nivel "correcto" universal — depende del riesgo del sistema.

APLICADO

De vuelta a Knight Capital

Con el vocabulario de hoy, dos momentos distintos del caso de la semana pasada tienen nombre propio:

- **Falla de verificación:** en 2005 se movió la función de cantidad acumulada y nadie volvió a revisar ni probar el código de *Power Peg* tras el cambio.
- **Falla de validación:** el 1 de agosto, el sistema desplegado no se comportó como el mercado —su usuario real— necesitaba, aunque cada pieza por separado "funcionara".

Un solo caso, dos tipos de falla distintos — encadenados.



SÍNTESIS

Lo que hay que llevarse de hoy

1. **QA no es QC**. Aseguramiento es preventivo y mira el proceso; control es detectivo y mira el producto.
2. **Verificación** pregunta si lo construimos correctamente; **validación**, si construimos lo correcto.
3. Se puede verificar con éxito y **validar con fracaso** al mismo tiempo.
4. El **modelo en V** empareja desarrollo y pruebas desde el inicio — útil con requisitos estables, cuestionable en entornos ágiles.
5. La calidad ya no es solo de "el área de QA": es **responsabilidad compartida** de todo el equipo.



INSTRUCCIONES

Taller de hoy: Matriz V&V

2 horas, en sala de Zoom asignada — grupo aleatorio de la semana

El docente entrega un proyecto narrado con **12 actividades** de su ciclo de vida (revisiones, pruebas, demos, aprobaciones).

Cada equipo debe:

- Clasificar cada actividad como **verificación** o **validación**.
- Justificar cada clasificación en una o dos frases, usando el vocabulario de hoy.
- Señalar, si lo hay, algún hueco: una fase de desarrollo sin su contraparte de prueba.

La entrega cierra al terminar las dos horas. Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 3, hay que llegar con esto leído:

- Historias de usuario y criterios de aceptación.
- Plan de aseguramiento de la calidad — SQAP, norma IEEE-730.

Los grupos siguen siendo **aleatorios** hasta la semana 5. El taller de la semana 3: escribir 5 historias de usuario con criterios de aceptación en **Gherkin**, sobre un dominio asignado por grupo.

